STUDY NOTE · NOT A BUILD SPEC

Which architecture fits FacilityCore, and which ones do not

A plain-language guide for someone learning system design. Every pattern is defined, then tried against this product: maintenance bills, UPI confirmation, and one database.

Product Phase 1 · one API, one database, many societies

Read this before choosing folders or libraries.

HOW TO READ THIS

You do not need these words yet

Architecture is the shape of the program. It is not the list of features. The business requirements already say what FacilityCore does: a society sends a bill, a resident pays the society's own UPI ID, and a treasurer confirms the bank reference. This note answers a different question. Where should that confirmation rule live, and which famous shapes would put it in the wrong place?

Read it in order. Each pattern is defined in ordinary language, then applied to Palm Court CHS, flat B-1204, and a bill of Rs 4,850. At the end there is a short test. If you can answer it without looking back, the concepts have stuck.

KEEP ONLY THIS PART

A pattern is a tool, not a rank.

Clean architecture is not "more advanced" than a simple script. CQRS is not "more professional" than a single database. The right shape is the smallest one that keeps the rules true. A bigger shape has a cost: more code, more ways for the balance to be wrong, and more to learn before you can change a late fee.

1. Five facts that choose the shape

Name a pattern only after the facts are written down. If a pattern fights one of these five, it is the wrong pattern for this product, even if it is famous.

	Fact
1	One program, one database, many societies. Every bill, payment, and complaint belongs to exactly one society. A person in Palm Court must not see Lakeview.
2	The next read must show the last write. When the treasurer confirms Rs 4,850, Anita's next open of the home screen must show nothing due. A delay of "a few seconds" is a wrong balance.
3	The money rules are strict and easy to get wrong. One bill per flat per month. The late fee pauses only while a payment is reported. Incoming money clears fees first, then the oldest bill. A payment row is never deleted.
4	Three doors, one set of rules. The resident phone, the committee phone, and the staff website all ask the same server. The late-fee formula must not be copied into three places.
5	One team, one release, phase 1. Later ideas (a gate, WhatsApp, a priced service) exist as switches that are off. They are not extra programs.

These five facts are the whole of the decision. The rest of the note is what they imply.

2. What the word architecture means

Think of a house. The business requirements are the rooms the family asked for: a kitchen, two bedrooms, a place to pay the bills. Architecture is which walls are load-bearing, and whether the kitchen plumbing can be replaced without demolishing a bedroom. Folder names and libraries are the furniture. Furniture can move. A load-bearing wall cannot, so you choose it carefully and you choose few of them.

In this API the load-bearing wall is the money rule. SMS, Excel export, and the phone screens are plumbing and paint. They may change. The rule "confirm writes one ledger line and stops the fee" must stay true while they change.

3. The patterns, defined once

People often stack these words as if you must pick one brand. They answer different questions. You can use a small piece of one inside another. You should not adopt the largest version of each just because the name came up.

Layered, also called n-tier

The oldest split: a screen layer, a rules layer, and a database layer. Requests travel downward. This is the right first picture. Its weakness is that the middle layer slowly learns SQL, HTTP, and SMS, until "the rules" and "the database" are the same class. Clean architecture is layered design with a stricter rule about who may know whom.

MVC — model, view, controller

A way to split one screen. The view draws. The controller receives the click. The model holds the data. Our phones are the view. The API has no HTML pages. MVC does not say where the late-fee formula lives. Use MVC inside a website that renders pages. Do not call the FacilityCore server "an MVC app" and think the architecture question is finished.

Clean architecture

Draw the program as rings. Dependencies point inward only. Inner rings do not import outer rings.

Ring	Name	In this product
Innermost	Entities	Bill, payment, ledger line, the late-fee formula. No HTTP. No database. No SMS. A unit test can run the Rs 3,015.03 check with nothing else installed.
Next	Use cases	"Confirm this payment." "Send the March bills." They call entities and ask an interface to save. They do not know whether the save is Postgres or a test double.
Next	Adapters	The HTTP endpoint, the database mapping, the SMS sender. They translate between the outside world and the use case.
Outside	Frameworks	ASP.NET Core, the database engine, the SMS provider. Replaceable without rewriting the fee formula.

The dependency rule, in one sentence: the class that computes the fee is not allowed to mention a controller or a table.

> **USE THIS HERE**
>
> **Clean architecture is the shape of this API.**
>
> Three clients share one set of money rules, and those rules must be testable without starting the website. That is exactly the problem the rings were drawn to solve. It would be the wrong shape for a one-evening form that only stores a name.

Domain-driven design (DDD)

DDD is two different sizes of idea, and mixing them up causes the confusion.

Tactical DDD is a handful of techniques for code that has real rules.

• **Ubiquitous language.** The word in the meeting is the word in the code. "Reported" means the resident has typed the bank reference and the treasurer has not decided. It does not also mean "paid".
• **Aggregate.** A cluster of rows you save in one go, so they cannot be half-updated. Confirming a payment writes the payment, the ledger line, and the allocation together, or it writes none of them.
• **Invariant.** A rule that is always true when the save finishes. Example: a flat has at most one bill for March.

Strategic DDD is for an organisation: separate models, separate teams, maps of how those teams talk. "Payment" in maintenance and "provider payment" for a future plumber are already two meanings. That is a small strategic insight, and we honour it by keeping provider money off the maintenance ledger. We do not create a Billing Department service and a Complaints Department service.

KEEP ONLY THIS PART

Keep the language and the payment aggregate. Leave the organisation chart.

The money cluster is rich enough to deserve a name, a boundary, and tests. Complaints, notices, and polls are records with a few guards. Giving each of them a full aggregate ceremony, or its own database, does not make the product safer.

CQRS — command query responsibility segregation

A command changes something and usually returns little. A query reads something and changes nothing. That sentence alone is just tidy code. ConfirmPayment is a command. GetStatement is a query. You may name methods that way inside clean architecture. That is not "adopting CQRS".

The architecture called CQRS goes further. The write model and the read model are different, often stored apart, and the read model is updated by a message after the write. The read is allowed to be a moment behind. That trade is worth it when millions of people read a catalogue and the stock figure can be a few seconds old. It is a bad trade when the figure is someone's dues.

DO NOT USE THIS HERE

A separate read model would let the balance lie.

Anita is waiting on Rs 4,850. The treasurer confirms. If "amount due" lived in a second table that updates when a message arrives, her phone can still show Rs 4,850 after the money has been accepted. The dashboard, the statement, and the defaulter list are sums of the same ledger. They must be that sum, not a copy of it.

Event sourcing

Store every change as an event, and rebuild today's state by replaying the events. Use it when you must reconstruct many different past views from one stream, and you accept the extra machinery. FacilityCore already has that stream: the ledger. Opening, charge, fee, payment, waiver, bounce, and refund are the events, and the balance is their sum. A second event log beside the ledger would record the same history twice and give you two places to be wrong.

DO NOT USE THIS HERE

The ledger is the history. Do not add another one.

A payment is never deleted. A mistake is a reversing line. That is already an append-only journal. Event sourcing on top of it duplicates the journal.

Microservices

Many small programs, each with its own database, talking over the network. Use them when separate teams must release on separate days and the boundary between them has been stable for a long time. The cost is that one user action can succeed in one program and fail in the next. "Confirm" would then be: billing service, then ledger service, then receipt service. If the receipt service is down, the fee might have stopped and the receipt might not exist. Our rule is that confirm is one consistent change.

The name for the shape we do want is a **modular monolith**: one program you deploy, with modules inside it, one database, and one transaction around a confirm. Modules can still be strict about not reaching into each other. That strictness is clean architecture. It does not require a network hop.

USE THIS HERE

One API. Modules inside it. One database transaction around money.

Complaints do not need their own server. A society id on every row is the boundary that matters in phase 1. Split a service out only after a real pressure appears: a different scaling need, or a team that must ship on its own calendar. That pressure is not in this phase.

4. One action, walked through the rings

Follow a single confirm. This is the picture to keep. If a proposed design cannot tell this story without a second database or a message bus, the design is larger than the problem.

Step	What happens
1. Door	Anita's phone and the treasurer's phone are outside the rings. They send HTTP. They do not contain the fee formula.
2. Adapter	The endpoint reads the token. The token says treasurer, society Palm Court. A secretary token is refused here, because that role may not confirm. The endpoint then calls the use case. It does not compute the allocation itself.
3. Use case	"Confirm payment for reference 123456789012." It loads the reported payment and the flat's open bills through an interface. It does not know the SQL.
4. Entity	The payment may be confirmed only if it is still Reported. The allocation is fees first, then the oldest bill. The ledger line is negative (the flat owes less). The fee stops. These sentences are code in the entity, and a test can run them with Rs 4,850 and no database.
5. Save	The adapter writes payment, ledger line, and allocation in one database transaction. Then it may send a receipt message. If the save fails, nothing is half-written.
6. Next read	Anita opens home. A query sums the same ledger. The sum is zero. There is no copied "amount due" column to forget to update.

Notice what is absent. No event is published to rebuild a dashboard. No second service owns the receipt. The staff website, on another day, creates a society through the same style of use case, and it is refused if it tries to call confirm. Same rings, different role.

5. Side by side, for this product only

The verdict column is about FacilityCore phase 1. The same pattern can be the right answer on a different product. Section 7 is how you tell.

Pattern	What it would do here	Verdict
Layered n-tier	A start. Becomes unsafe if the middle layer speaks SQL.	Use the idea. Tighten it into rings.
MVC	Does not place the fee formula.	Wrong question for this API.
Clean architecture	Rules in the centre, HTTP and database outside, three clients share the centre.	This is the shape.
DDD, tactical	One language, one aggregate around confirm.	Use on money. Not on every screen.
DDD, strategic	Separate models and teams.	Too big. One team, one database.
CQRS, as method names	Confirm is a command, statement is a query.	Harmless naming. Optional.
CQRS, separate stores	Dashboard updated later from events.	Breaks fact 2. The balance can lie.
Event sourcing	A second history beside the ledger.	The ledger already is the history.
Microservices	Confirm split across programs and databases.	Breaks the single transaction.
Modular monolith	One deployable, modules, one database.	This is the deployment shape.

6. What "someone suggests it" should sound like

You will hear these in reviews. A clear reply is part of knowing the design. Each reply uses a fact from section 1, not a slogan.

"Complaints should be their own microservice."

Reply: a complaint is a row on a flat, seen by the secretary and the facility manager. It does not have its own release calendar, and it must not be visible across societies. A module inside this API, filtered by society id, does that. A second program would add a network failure to "mark resolved" and would not remove any real coupling. Revisit this only if complaints later need to scale, or ship, independently of billing. They do not today.

"Put the dashboard on CQRS so reports stay fast."

Reply: the dashboard is collected, pending, overdue, late fees, and the amount still awaiting confirmation. Those are sums and filters on the ledger and on reported payments. If they become slow, the first tools are an index and a narrower query, still reading the ledger. A copied total is faster only until it disagrees with the ledger, and a disagreement is a wrong dues figure. Fact 2 forbids a stale copy.

"Event-source the ledger. It is already events."

Reply: the insight is right and the extra system is not. The ledger rows are the events, and the balance is the fold of those rows. We already refuse to edit or delete them. Rebuilding state from a second log would mean two journals that can diverge. Keep one journal.

"Use DDD everywhere so the design is consistent."

Reply: consistency of language, yes. A ceremony per screen, no. "Reported", "bill payer", and "window" must mean one thing in the fee code and in the requirements. A notice is a title, a body, and a language. It does not need an aggregate root, domain events, and a repository interface of its own unless a rule appears that can be broken by a partial save. Spend the strictness where a partial save costs someone money.

7. A method you can reuse on the next system

Ask these in order. Stop at the first answer that picks a shape. Do not collect patterns.

	Question	If the answer is...	Then
1	Must the next read show this write immediately?	Yes, and the read is the same numbers.	One database. No CQRS read model. No "eventual" dashboard.
1b	Same question, other answer.	Yes for money, but a catalogue page can be seconds behind.	CQRS only on that catalogue. Leave money on the write database.
2	Can this action be half-done and still be honest?	No. Confirm is all or nothing.	One transaction. Not microservices for the steps inside confirm.
3	Are the rules easy to get wrong, and will they outlive the UI?	Yes. The paisa test must pass after the screens change.	Put the rules in an inner ring with no web framework. That is clean architecture.
4	Do two teams need to deploy on different days?	No.	One program. Modules, not services.
5	Does one word mean two different things?	Yes. Maintenance payment and a future provider payment.	Two types, no shared ledger line. That is the useful bit of strategic DDD.

Run the method on a different product in your head. A public transit board with a million readers and a timetable that can be a minute old answers question 1 the other way: a read model is reasonable. A bank transfer answers it the way FacilityCore does. The pattern follows the answer. The answer does not follow the pattern you most want to learn.

8. Where this sits next to the code you already have

The repository currently has a .NET 10 Web API template at src/FacilityCore.Api. Its weather endpoint is the template's sample. It is not the architecture, and it is not a FacilityCore feature. When the real endpoints arrive, a controller that computes a late fee inline has already broken the rings, even if the number happens to be right today. The fee belongs in the inner ring so the day-10 growing check, Rs 3,015.03 on a Rs 3,000 principal at 0.05% a day, can run as a test with no HTTP server.

Folder names are furniture. They should make the rings obvious: something that holds entities, something that holds use cases, something that holds HTTP and the database. The exact tree can follow a layout you choose. The dependency rule does not change with the folder names. An

entity project that references the web project is the rule broken, whatever the folders are called.

9. Check yourself

Answer in a sentence each. The answers are underneath, so try before you read them.

1. Why is the treasurer's confirm not allowed to update a separate "amount due" row and finish?

Because the home screen must show the ledger sum, immediately. A second row can be forgotten, and for that moment the resident owes the wrong amount.

2. Name the one cluster that deserves an aggregate.

The payment together with the ledger line and the allocation. They save together or not at all. A notice does not need the same treatment.

3. What is the difference between "we have a Confirm method and a GetStatement method" and CQRS?

The first is naming. CQRS as an architecture means a different read model, usually updated later. We want the naming at most, and we refuse the second model.

4. Why is the ledger not a reason to adopt event sourcing?

It is already the append-only history. The balance is the sum of its lines. A second log would duplicate it.

5. A classmate says microservices are more scalable. What fact answers them?

Fact 2 and fact 5. Confirm must be one transaction, and nothing in phase 1 needs its own deployment. Scale a slow query first. Split a program only when a team or a load forces it.

6. Which ring is allowed to import ASP.NET?

The outer adapter. The entity that computes the fee is not.

7. Why can the staff website and the resident phone share a server without sharing a screen?

They are two adapters over the same use cases. The token's role decides which use case will even run. System admin never reaches confirm.

8. When would CQRS become the right answer inside this product?

Only for a read that is allowed to be stale and is too heavy to compute from the ledger. Dues, the statement, and overdue are not that read. A future public catalogue might be.

10. The sentence to keep

> **USE THIS HERE**
>
> **One program, rings pointing inward, one ledger, one transaction around confirm.**
>
> Clean architecture is the shape. Tactical domain-driven design is how the money cluster is written. CQRS, event sourcing, and microservices answer problems this phase does not have. Learn them so you can recognise those problems later. Do not install them in advance.

The build rules, the role table, and the paisa test live in docs/brd-product-phase-1.md. This note does not replace them. It only explains why the server is shaped the way it is.